USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

Touchscreen, and a Verification Method That Failed Three Times

UM-NATOS-017

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-017	1.2 · 2026-08-15	**Complete, verified on hardware** — §4.1 and §7.1–7.3 added after the launcher exposed an inverted axis	Internal

1. Abstract

The CYD's XPT2046 resistive touchscreen works: contact is detected through the controller's PENIRQ line, and both axes are mapped to panel coordinates by measurement rather than by reasoning. **§8 (revision 1.1)** carries it up to the applications: bytecode can now read the pointer, confined to its own viewport, which makes the system interactive for the first time.

The driver took four attempts, and only one of the four failures was in the driver. Two were in `gpio.h`, and the fourth — the one worth reading this report for — was in **how the results were being read**. §5 documents a serial capture that rebooted the board and erased the evidence before printing it, and the two conclusions recorded as fact on the strength of it.

This is the third time in two sessions that an instrument reported its own reading invalid and the number beside it was believed anyway. §6 treats that as the finding.

Revision 1.2 records what a real consumer found. The horizontal axis had been **inverted since the driver was written**, and the calibration in §7 not only missed it but concluded the opposite — because it inferred direction from the last sample of a drag, which is the one sample guaranteed to be invalid. The same sample class was also feeding position readings to every consumer, fixed in §4.1 by gating on pressure. Neither was found by inspection; both were found by building something that depended on horizontal position for the first time.

2. Hardware

Signal	GPIO	Note
CLK	25	
MOSI	32	second GPIO bank
MISO	39	input-only pin, second bank
CS	33	second bank
IRQ (PENIRQ)	36	input-only pin, second bank

Pins and calibration extremes are the board's, taken from the vendor driver.

A separate bus from the display, which matters. The display driver leaves CS asserted between a window command and its pixel stream (UM-NATOS-015 §4), so a shared bus would mean either interleaving touch traffic into an open window or serialising every touch read behind a 387 ms full-screen fill. Neither is acceptable for a pointer.

Bit-banged, for the same reason the display is: it needs only GPIO registers, so a failure can only be the panel, the pins, or the protocol.

3. Two defects that made a broken driver look plausible

3.1 `gpio.h` only ever handled pins 0–31

Pins 32–39 have entirely separate registers — `OUT1`, `ENABLE1`, `IN1` — and **nothing complains when the wrong bank is used**. A shift of 39 on a 32-bit register is undefined and reads zero in practice.

So MISO on GPIO 39 read a constant 0, and MOSI and CS on 32 and 33 were never driven at all. Every accessor now selects its bank from the pin number, so a caller never has to know the split exists.

This bug was latent from the moment `gpio.h` was written for the display, whose pins are all below 32. It would have broken **any** peripheral above pin 31.

3.2 Silence read as maximum pressure

Pressure was computed as `z = z1 + 4095 - z2`. A controller returning zeros on both channels yields **4095** — full scale. A dead bus therefore presented as a permanent hard press, and the first run reported `s/e=10/10`: every sample a touch, on a screen nobody was touching.

An all-zero reading is now rejected outright. The general point is worth keeping: **a formula whose failure mode is a plausible extreme is worse than one that fails to an obvious value**, because the first requires interpretation and the second announces itself.

3.3 The first conversion is the one that matters

The control bytes originally set `PD1:PD0 = 00`, powering down the ADC and its reference between every conversion. The first conversion after power-up is specified as inaccurate — and `Z1`, the channel that decides whether a touch exists, is the first conversion after CS goes low on every single read.

The burst now uses `PD=11` to keep the reference powered, with a throwaway conversion absorbing the inaccurate reading, and a final `PD=00` to power down and re-enable PENIRQ.

4. Detection: PENIRQ, not pressure

Both routes work. Measured under a real touch:

```
irq=17   z1max=1123   z2min=2992   zmax=2065
```

PENIRQ asserted 17 times, matching the event count exactly, and computed pressure peaked at 2065 against a threshold of 300.

PENIRQ is used because it is a direct electrical indication from the panel, needing no ADC, no threshold and no calibration. It is read *before* CS is asserted, since the pin is only meaningful while the controller is idle.

Pressure is still recorded despite no longer being the detector. A channel that never moves is itself evidence, and hiding it would only make the next person repeat the measurement.

4.1 PENIRQ answers a different question than it was asked

Added in revision 1.2, after the launcher.

The reasoning above is sound and the conclusion was still wrong, which is an uncomfortable combination worth stating precisely.

PENIRQ reports whether a finger is **there**. It says nothing about whether the position sample taken alongside it is **valid**. Those are different questions, and this section answered the second by measuring the first.

The pin asserts on approach and stays asserted through release. During both, the panel is not resistively bridged, so the position channels float and read their ADC rail. Measured on the launcher, within a single tap:

```
z=7     raw_x=4095   ->  x=0     approach, input floating
z=2025  raw_x=1280   ->  x=167   the actual finger
z=20    raw_x=4095   ->  x=0     release
```

A factor of a hundred separates the populations, and `Z_THRESHOLD` — 300, defined in `touch.c` and unused since it was written — sits cleanly between them.

The consequence is specific and was invisible for as long as nothing depended on horizontal position: **4095 is near the top of the range, so a rail reading maps to one edge of the screen.** Every spurious sample votes for the same place. In the launcher that place was the leftmost column, and it looked exactly like a broken axis rather than a valid-sample problem.

`out->down` is now `pen && (z > Z_THRESHOLD)`. PENIRQ still gates it, because a pressure reading taken while nothing is near is its own kind of noise; pressure now gates it too, because presence is not validity.

5. The capture was destroying its own data

5.1 Symptom

Three consecutive runs, across different builds and different user actions, reported **exactly 150 samples**.

A counter that lands on an identical value every time is not accumulating. It is measuring a fixed window.

5.2 Cause

Opening a serial port asserts DTR/RTS, which is wired to the ESP32's reset and boot pins. Configuring them *after* opening still glitches EN. Every capture described as "no reset" was rebooting the board, waiting for it to boot, and then reading roughly twelve seconds of a system nobody was touching.

Setting `dtr` and `rts` **before** `open()` fixes it. The same read then reports 647,406 samples and no boot banner.

5.3 What it invalidated

Two statements were recorded as findings on data that had already been erased:

- "*pressure never exceeds 1 under a firm press*" — actual value **1123**
- "*PENIRQ never asserts across two full drags*" — actual count **17**

Both were measured on a board that had rebooted seconds earlier. The `PD=11` fix of §3.3 had already worked when it was declared a failure.

Worse, the user reported that dots were following their finger, and that report was overridden as "almost certainly leftovers" on the strength of the broken measurement. **The human observation was the better evidence.**

5.4 Why it was hard to see

The reboot was invisible in every way that was being checked. Output looked normal, because a booting kernel prints a full banner and then healthy report lines. Values looked plausible, because they were real readings — of the wrong interval. Only the sample counter carried the contradiction, and it was printed in the same line as the values that were being trusted.

5.5 It happened again, with new tools

Added in revision 1.2.

Investigating the inverted axis needed a way to read board state while a finger was on the glass. Two scripts were written for it, both described as "no reset", and **both reset the board** — on Windows the serial driver asserts DTR on `open()`, and DTR drives EN.

So the same failure this section documents was reproduced, by someone who had just read this section, in tools written specifically to avoid it. Two rounds of four-corner taps were destroyed between being recorded and being read.

The fix is the one already written here: set `dtr` and `rts` **before** `open()`. What is new is the verification — uptime is now sampled twice across a connection and confirmed to increase, rather than the absence of a reset being assumed. Knowing the failure mode was not enough; a check was needed.

6. The finding

Three times across two sessions, an instrument reported its own reading invalid and the value beside it was believed anyway:

Instrument	What it said	What was believed instead
Marker block (UM-NATOS-016 §3.4)	Frozen — the system is halted	The colour bars, which a frozen screen renders identically
`s/e=150`, three runs	This counter is not accumulating	The pressure values printed beside it
Boot banner in the capture	The board just restarted	The counters, as though continuous

The peripherals were largely fine. The verification method was what kept failing, and it failed in the same shape each time: **a signal that the measurement was invalid, sitting next to a number that looked reasonable.**

What broke the deadlock, every time, was the same move — **latch the quantity so timing cannot lie about it**, then feed the system a controlled input rather than interpret an uncontrolled one. §7 is that method applied deliberately.

7. Calibration by controlled input

Watching a trail follow a finger can say "*the dots go the wrong way*". It cannot distinguish a **swapped** axis from a **flipped** one, and several build cycles were spent guessing between those two before that was noticed.

Instrumenting the raw span of each channel answers both at once. Two drags, each along one screen axis:

Drag	`rx` span	`ry` span
Left → right	**486–3536** (3050)	2499–2862 (363)
Top → bottom	675–1518 (843)	**432–3204** (2772)

The mirror image is the result. Each drag swept one channel across most of its range while the other barely moved, which is precisely what a correct axis assignment looks like.

Direction falls out of the endpoints: the horizontal drag ended at `rx=3527` against a maximum of 3536, so raw X increases left to right. The vertical drag ran from `ry=586` to `ry=2171`, so raw Y increases downward. Neither axis needs a flip.

The original code had the axes transposed, on the reasoning that a portrait display must swap a landscape-calibrated panel. **That reasoning was wrong**, and no amount of looking at the screen would have said so.

7.1 The direction test was decided by its worst sample

Added in revision 1.2. The conclusion above about direction is wrong, and the method that produced it is why.

Raw X does not increase left to right. It decreases. Measured by tapping four labelled corners:

corner	`raw_x`	`raw_y`
top-left	**3360**	416
top-right	**591**	376
bottom-left	3258	**3518**
bottom-right	376	**3462**

Left reads ~3300, right reads ~480. The axis assignment in the table above is correct — `raw_x` really is horizontal — but its **sign** is backwards, and has been since the driver was written.

The span measurement was never the problem. The direction inference was:

> *"the horizontal drag ended at `rx=3527` against a maximum of 3536, so raw X increases left to right"*

That reads the **last sample of a drag**, which is the release sample — and §4.1 establishes that release samples read the ADC rail. A rail reading is 4095, near the top of any range, so a drag that ends anywhere at all "ends near its maximum", and **every axis appears to increase.** The test could only ever return one answer.

The same corrupted sample later decided the launcher's icon selection, three months apart, in a different file. One defect, two symptoms.

7.2 Endpoints are not the instrument; labelled points are

A drag produces a cloud of samples whose first and last are the two least trustworthy, because both sit at a contact transition. Inferring direction from precisely those two is the method's flaw, not a slip in applying it.

Four **labelled** taps have no such moment. Each is a known screen position paired with a raw reading, and the direction of every axis falls out of comparing labels rather than trusting an endpoint. It also yields the range and the sign from one measurement instead of two.

The corner table above is now recorded in `touch.c` as the calibration itself, rather than the two derived limits it produces. A future re-calibration can check the derivation instead of repeating the guess.

7.3 Why nothing noticed for three months

A backwards horizontal axis had no consumer:

- the application strips are full width, so horizontal position never mattered
- the raycaster steers from the left and right thirds of the view, so a reversed axis makes it turn the other way — which reads as a control preference
- §8's containment tests check that a touch is confined to the right viewport, which is a question about `y`

The launcher is the first thing in this project that asks **where across the screen** a finger is. It failed on the first tap.

7.4 The corners were the wrong four points

§7.2 is right that labelled points beat endpoints, and it still chose the four places on the panel where a labelled point is least trustworthy.

A finger cannot reach the extreme corner of a bezelled panel. Every corner reading was therefore short of the true extreme, the derived range came out narrower than reality, and — because the range is the *divisor* — every raw step counted for too many pixels. The mapping magnified position about the centre of the screen.

Measured against a proper fit, the corner constants were **23% over-sensitive on X and 11% on Y**:

	corner-derived span	measured span	error
X	3000	3694	23%
Y	3140	3484	11%

The consequence is what made it hard to characterise. The error is proportional to distance from centre **and changes sign**:

true screen x	old mapping returned	error
30	~1	−29 px
120	~112	−8 px
210	~222	+12 px

UM-NATOS-022 §3.2 described this as "about 24 px low on X", which is a fair reading of one symptom at one place on the screen and is not what was wrong. No fixed offset could have corrected a magnification, which is why every attempt to nudge the constants moved the fault somewhere else rather than removing it.

The fix is `kernel/calib.c`: four targets inset 30 px from the edges, tapped on the device. An inset target is somewhere a fingertip can actually go, so the reading is a real position rather than the closest approach to an unreachable one, and the mapping is interpolated outward to the edges instead of extrapolated inward from a guess.

Two properties of the routine matter more than the arithmetic:

- **The pairs are checked before anything is fitted.** Two targets share each screen coordinate, so every reading has a partner it should nearly match (tolerance 900 of a ~3000 span). The first run on hardware was refused by this check — two targets had been tapped on the wrong cross, and 3453 and 353 average to 1903, an entirely ordinary-looking number that measures nothing. Without the pair check the run failed its *later* sanity test and reported "readings too close together", which was true of the averages and described nothing that had happened.
- **The outcome is held, not just printed.** The result line is emitted the instant the fourth target is tapped, which is precisely when no capture is attached. That is §5's failure in a new costume — the third time in this project a measurement was written into a stream nobody was reading — so `calshow` reports the four readings, the outcome, and the calibration in use, and can be asked at any later time.

The result is persisted in the boot record (UM-NATOS-018, record version 3) and restored at startup, verified across a hard reset:

```
touch cal    : restored x 141..3835 y 243..3727
```

`touch_set_calibration()` refuses a degenerate range, and the routine reports what was **read back** rather than what it computed, so a refusal cannot be mistaken for an install.

8. Input reaches applications

Added in revision 1.1. `SYS TOUCH` (8) returns `r0 = touched, r1 = x, r2 = y`.

8.1 Confinement

Coordinates are **viewport-relative**, and a touch anywhere else on the panel is reported to the asking application as **no touch at all**.

That is stronger than refusing to answer. An application is never told where its viewport sits — `SYS DIMS` returns size only — so it cannot reconstruct an absolute position even from a touch it is permitted to see, and

it has no way to learn that a touch happened elsewhere. Input joins memory and pixels as something a program cannot observe outside its own allocation:

Resource	Confined by	Outside its allocation
Memory	Arena bounds check	Unrepresentable — an address outside the arena cannot be formed
Pixels	Viewport clipping	Clipped before it exists
Input	Viewport test on delivery	Reported as no touch

Comparison is in the offset domain, like `arena_contains()` and `vp_fill()`, so a touch above or left of the viewport underflows to a huge unsigned value and is rejected rather than wrapping into range.

8.2 Measured

```
touch g/w = 81/109211        last = ...->191,174
```

81 touches delivered to the paint application and **109,211 withheld** for landing outside its 168–194 strip, with the last accepted touch at y=174 — inside it.

Both halves matter. Delivery alone would not show confinement, and withholding alone would be indistinguishable from a broken syscall. The withheld count is large because the application polls continuously: it asked for the pointer tens of thousands of times while a finger was demonstrably on the glass elsewhere, and was told there was none every time.

8.3 A snapshot, not the bus

The syscall reads a state published by the polling task rather than driving the controller itself. Doing its own SPI would cost milliseconds per call and contend with the touch task for the bus.

It is therefore cheap, and unlike `FILL` and `TEXT` it does **not** end the time slice (UM-NATOS-016 §2.4). The distinction is the cost of the work, not whether it is a syscall.

8.4 Two defects this exposed

Launching by index. `PROGRAMS` grew from three entries to six, and a hard-coded `PROGRAMS[4]` silently stopped meaning `paint` and started meaning `gfxrogue` — so boot launched the panel-flooding rogue instead of the interactive application. The symptom was a strip flickering red and white, with no visible connection to an array index. Boot now launches **by name**.

The user's description — "*touching in the red/white thing was making black dots*" — identified both the wrong application and the cursor artefact, and was a faster route to the cause than the counters, which correctly reported `g/w = 0/0` without indicating why.

The kernel was scribbling on application viewports. A kernel-drawn touch cursor painted over whatever strip the finger was in — exactly what the kernel forbids applications from doing to each other. Not unsafe, since the kernel is trusted, but wrong about ownership: those pixels belong to whoever owns the strip. It also destroyed what it drew over, replacing an application's output with the black squares it left while erasing its own previous position.

The cursor is removed. An application that wants pointer feedback draws it itself, through `SYS_TOUCH` and `SYS_FILL`, in its own coordinates and inside its own viewport — which is the entire point of the syscall.

9. Metrics

Quantity	Value
Detection	PENIRQ, GPIO 36
Poll rate	~100 Hz (once per tick)
Conversions per read	11, including one throwaway
Peak pressure measured	2065 (threshold 300)
`z1` under touch	1123, against 0 idle
`z2` under touch	2992, against ~4090 idle
Raw span, horizontal drag	3050 counts
Raw span, vertical drag	2772 counts
Axis flips required	0
Native tasks	10
Image size	21,888 B
Attempts before working	4
Conclusions retracted	2
Touches delivered to an application	81
Touches withheld as out-of-viewport	109,211
Confinement failures	0
`raw_x` at the left edge	~3300
`raw_x` at the right edge	~480
Direction of `raw_x`	**decreases** left → right
Rail samples per tap, before the pressure gate	~2 of 3
Pressure, real contact vs approach/release	~2000 vs ~10
`Z_THRESHOLD`	300 — defined at the start, first used in revision 1.2
Months the X axis was inverted	~3
Tools written to avoid §5's failure that reproduced it	2 of 2

10. What this does not establish

- **No multi-touch.** A resistive panel cannot report two points; a second finger produces a reading somewhere between them.
- **No IRQ-driven wakeup — but no longer for want of an interrupt.** PENIRQ is now routed through the interrupt matrix to CPU line 23 and its handler runs (UM-NATOS-023 §6). What has never been observed is a *finger* causing that to happen, so the touch task still polls at 100 Hz and the consumer is switched off (§7 there). The gap moved from "not wired" to "wired and unexplained", which is progress of a sort and is not the same as working.
- **No debounce or gesture recognition.** Every sample is independent; there is no tap, hold, drag or double-tap concept.
- **No pointer-up event.** An application sees only that a touch is or is not present. There is no press, release or tap, so it cannot distinguish a held finger from a rapid sequence of contacts.
- **One pointer, shared.** All applications read the same snapshot. Two applications whose viewports overlapped would both see the same touch; nothing arbitrates focus, because nothing yet needs to.
- **The calibration is this board's, but only this board's.** §7.4's pass has been run on one unit, with one finger, once. The vendor extremes remain the compiled-in defaults for a panel that has never been calibrated, and they are known to be wrong by 23% on X — a first boot is usable, not accurate.
- **The cursor leaves a black square** where it has been, because nothing records what was underneath. Fixing that needs either a panel read-back this hardware does not reliably support, or damage tracking the display layer does not have.
- **The calibration is still linear.** §7.4 fits four inset targets, averaging two readings per axis, and the residuals were small (16 and 174 raw on X, 145 and 2 on Y). But a linear map cannot express panel non-linearity, and the two targets on each axis sit at the same two positions — so nothing measures the error *between* them. A bilinear or per-corner fit would; this does not.
- **Nothing detects that a calibration has gone stale.** A restored calibration is trusted indefinitely. If the panel ages, or a different unit boots this flash image, the stored numbers are wrong and nothing says so.
- `Z_THRESHOLD` **is a single number chosen from one measurement.** 300 sits between populations that were ~10 and ~2000 on this panel, on this day, with this finger. A lighter touch has not been tested, and a threshold that rejects a real press is worse than one that admits a bad sample.
- **No filtering beyond the gate.** Samples that pass the pressure test are used as they arrive. There is no median-of-N or hysteresis, so a valid-pressure outlier still lands wherever it says.
- **Nothing re-checks the axis at runtime.** A different panel, or the same one bonded the other way, would need this rediscovered by hand.

11. References

- UM-NATOS-015 — the display driver, and the shared-bus problem avoided here
- UM-NATOS-016 §3.4 — the frozen marker, the first instance of the §6 pattern
- `kernel/gpio.h` — the two-bank split of §3.1
- `kernel/touch.c` — PENIRQ detection, PD handling, and the measured mapping